Part 3: Neural Transition-Based Dependency Parsing

3(a): Transition sequence

The stack is shown from bottom to top. Each row shows the configuration **after** applying its transition. Dependencies point from head to dependent.

Step	Stack	Buffer	New dependency	Transition
0	[ROOT]	[I, presented, my, findings, at, the, NLP, conference]	—	Initial
1	[ROOT, I]	[presented, my, findings, at, the, NLP, conference]	—	SHIFT
2	[ROOT, I, presented]	[my, findings, at, the, NLP, conference]	—	SHIFT
3	[ROOT, presented]	[my, findings, at, the, NLP, conference]	presented → I	LEFT-ARC
4	[ROOT, presented, my]	[findings, at, the, NLP, conference]	—	SHIFT
5	[ROOT, presented, my, findings]	[at, the, NLP, conference]	—	SHIFT
6	[ROOT, presented, findings]	[at, the, NLP, conference]	findings → my	LEFT-ARC
7	[ROOT, presented]	[at, the, NLP, conference]	presented → findings	RIGHT-ARC
8	[ROOT, presented, at]	[the, NLP, conference]	—	SHIFT
9	[ROOT, presented, at, the]	[NLP, conference]	—	SHIFT
10	[ROOT, presented, at, the, NLP]	[conference]	—	SHIFT
11	[ROOT, presented, at, the, NLP, conference]	[]	—	SHIFT
12	[ROOT, presented, at, the, conference]	[]	conference → NLP	LEFT-ARC
13	[ROOT, presented, at, conference]	[]	conference → the	LEFT-ARC
14	[ROOT, presented, conference]	[]	conference → at	LEFT-ARC
15	[ROOT, presented]	[]	presented → conference	RIGHT-ARC
16	[ROOT]	[]	ROOT → presented	RIGHT-ARC

3(b): Number of steps

Parsing a sentence of n words takes **2n transitions**: each word is shifted onto the stack once, and each of the n arc transitions removes one word, leaving only ROOT.

3(c): Transition mechanics

Implemented in `parser_transitions.py`, in `PartialParse.__init__` and `PartialParse.parse_step`. The buffer is a copy of the sentence so parsing preserves the input list. Dependencies are stored as `(head, dependent)` tuples.

3(d): Minibatch parsing

Implemented in `parser_transitions.py`, in `minibatch_parse`. Partial parses retain input order in the original list; a separate list tracks unfinished parses. The model predicts one transition per active parse in each minibatch, and completed parses are removed from the unfinished list. Empty sentences are already complete.

3(e)(i): Derivative of the hidden activation

Let $z = xW + b_1$, so $z_i = \Sigma_j x_j W_{ji} + b_{1i}$ and $h_i = \max(z_i, 0)$. By the chain rule,

$\partial h_i/\partial x_j = (\partial h_i/\partial z_i)(\partial z_i/\partial x_j)$

$= W_{ji}$ if $z_i > 0$, and 0 if $z_i < 0$.

The case $z_i = 0$ is omitted as allowed in the question.

3(e)(ii): Derivative of cross-entropy with respect to a logit

With true class c, $y_j = 1$ when j = c and 0 otherwise. Since $\hat{y}_j = \exp(l_j)/\Sigma_k \exp(l_k)$,

$\partial \hat{y}_j/\partial l_i = \hat{y}_j(\delta_{ji} - \hat{y}_i)$,

where δ_{ji} is 1 when j = i and 0 otherwise. Applying the chain rule to $J = -\Sigma_j y_j \log(\hat{y}_j)$,

$\partial J/\partial l_i = -\Sigma_j (y_j/\hat{y}_j)\, \hat{y}_j(\delta_{ji} - \hat{y}_i)$

$= -y_i + \hat{y}_i \Sigma_j y_j$

$= \mathbf{\hat{y}_i - y_i}$.

Thus the derivative is $\hat{y}_i - 1$ for i = c, and $\hat{y}_i$ for $i \neq c$. For a minibatch mean loss, each example's logit gradient is additionally divided by the batch size.

3(e)(iii): Implementation and measured results

`parser_model.py` implements vectorized embedding lookup, the two affine transformations using explicit trainable parameters, ReLU, and dropout after ReLU. Its output is raw logits. `run.py` implements Adam, mean cross-entropy, backpropagation, and parameter updates. Neither `nn.Linear` nor `nn.Embedding` is used.

Training uses the starter defaults: 36 features, 50-dimensional embeddings, 200 hidden units, dropout probability 0.5, batch size 1024, Adam learning rate 0.0005, and 10 epochs. The checkpoint with the highest development UAS is used for test evaluation.

Measured on a full, non-debug training run:

Metric	Result
Best development UAS	**89.03%** (epoch 10)
Test UAS using the best development checkpoint	**89.12%**
Final average training loss	**0.06598**

These are measured results, not estimates. Training used CPU execution with Python 3.14.2, PyTorch 2.14.1, and NumPy 2.5.4; installed versions differ from the original handout's environment. The model and hyperparameters follow the starter defaults. Runs may vary because the starter does not fix random seeds.

Running the code

From this assignment directory, using the local environment:

```
.venv/bin/python parser_transitions.py part_c
.venv/bin/python parser_transitions.py part_d
.venv/bin/python parser_model.py -e -f
.venv/bin/python run.py -d # optional smaller debugging run
.venv/bin/python run.py # full training and test evaluation
bash collect_submission.sh
```

The full training log is saved in `results/training.log`. The coding submission is `assignment2.zip`; this written file covers only Part 3.